

Agentic Content Generation System

Team Guide — Read This Before You Write Any Code

DEPI Graduation Project — AI & Data Science Track

PART 1 — EVERYONE READS THIS

1. What Is This Project

We are building a system that automatically generates high-quality written content. A user types a topic. Six AI agents work together: one researches the topic, one plans the structure, one writes it, one edits it, one polishes it, and one coordinates everything. Each of you owns one agent.

Topic → Research → Plan → Write → Edit → Polish → Final Content

NOTE: Agents never talk to each other directly. Every agent returns its result to the Orchestrator, which decides what happens next.

2. Who Does What

Member 1	Research Agent	Searches the web and collects information about the topic
Member 2	Planning Agent	Turns the research into a clear content outline
Member 3	Writing Agent	Writes the full content based on the outline
Member 4	Editing Agent	Reviews the content and requests improvements if needed
Member 5	Optimization Agent	Polishes the final tone, style, and length
Member 6	Orchestrator	Builds the full system skeleton and coordinates everything

3. How to Set Up Locally

Do this once when you first join. Ask Member 6 if anything fails.

Install the tools

1. Python 3.11+ → <https://python.org>
2. uv (package manager) → run this in terminal:
`curl -LsSf https://astral.sh/uv/install.sh | sh`
1. Docker Desktop → <https://docker.com/products/docker-desktop>
2. Git → <https://git-scm.com>

Get the project and run it

1. Clone the repo:
`git clone https://github.com/your-org/agentic-content-system`

```
cd agentic-content-system
```

1. Install dependencies:

```
uv sync
```

1. Copy the env file and add your API key (Member 6 gives you this):

```
cp .env.example .env
```

1. Install code quality hooks:

```
uv run pre-commit install
```

1. Start the system:

```
docker compose -f docker-compose.dev.yml up -d
```

1. Verify it works:

```
curl http://localhost:8000/health
```

You should see status OK. If you do — you are ready.

4. The Agent Contract

Every agent receives the same input and must return the same output. Follow this exactly.

What your agent receives:

```
job_id    - unique ID for this job
topic     - the topic the user entered
context   - dict with all previous agents outputs
config    - which model to use (do not touch)
```

What your agent must return:

```
job_id    - same ID you received
agent     - your agent name e.g. 'writing'
result    - your output as a string
metadata  - dict: model, tokens, latency_ms
status    - 'success', 'retry', or 'failed'
```

The only method you write:

```
class YourAgent(BaseAgent):
    async def execute(self, input: AgentInput) -> AgentOutput:
        result = await self.model.generate(your_prompt)
        return AgentOutput(
            job_id=input.job_id, agent='your-name',
            result=result,
            metadata={'model': ..., 'tokens': ..., 'latency_ms': ...},
            status='success'
        )
```

- ✓ Always return AgentOutput with all fields filled
 - ✓ Always use self.model.generate() to call the LLM
 - ✓ Always read previous outputs from input.context
 - ✗ Never call OpenRouter or any LLM library directly
 - ✗ Never hardcode your model name — it comes from input.config
 - ✗ Never crash — catch all errors and return status: failed
-

5. Git Rules

main	Production. Never touch this directly.
develop	Main working branch. All work ends up here.
feature/your-agent	Your personal branch. This is where you work.

Your daily workflow:

1. Pull latest changes:
`git checkout develop && git pull`
`git checkout feature/your-agent-name`
1. Make your changes and test them (see Section 6).
1. Commit — message format matters:
`git commit -m 'feat(writing): add outline-to-content generation'`
NOTE: Format: type(agent-name): description. Types: feat, fix, refactor, test, docs
1. Push and open a Pull Request to develop:
`git push origin feature/your-agent-name`

Hard rules:

- ✗ **Never push directly to main or develop**
 - ✗ **Never force push**
 - ✗ **Never modify another member's agent file**
 - ✗ **Never modify shared/schemas.py without asking Member 6**
 - ✓ **Always open a Pull Request — never merge directly**
-

6. How to Test Your Agent

Option 1 — ADK Web (easiest, use this first)

```
cd packages/agents && uv run adk web
```

Opens browser at `http://localhost:8000`. Select your agent, type a test input, see the output.

Option 2 — Run unit tests

```
uv run pytest tests/unit/test_your_agent.py
```

Option 3 — Full pipeline test

```
uv run pytest tests/integration/test_pipeline.py
```

Before every Pull Request — run both:

```
uv run pytest tests/unit/test_your_agent.py
uv run ruff check packages/agents/your-agent/
```

Both must pass with zero errors. CI will reject your PR otherwise.

PART 2 — YOUR SECTION — Find your member number and read only that section

MEMBER 1 — RESEARCH AGENT

Your job:

Search the web for information about the topic and save everything to the knowledge base so other agents can use it.

Your files — you only touch these:

- packages/agents/research/agent.py
- packages/agents/research/tools/search.py
- packages/agents/research/tools/scrape.py
- prompts/research.md — edit this to improve output quality
- tests/unit/test_research.py

What `execute()` must do:

1. Search the web using the search tool
2. Extract full content from the top results using the scrape tool
3. Summarize what you found
4. Save everything to LightRAG using `self.save_to_memory()`
5. Return the summary and sources in result

What to put in context:

```
context['research'] = {  
    'summary': 'short summary of findings',  
    'facts': ['fact 1', 'fact 2'],  
    'sources': ['url1', 'url2']  
}
```

Your model:

```
meta-llama/llama-3.3-70b-instruct:free
```

MEMBER 2 — PLANNING AGENT

Your job:

Take the research and turn it into a clear structured outline with sections and key points.

Your files — you only touch these:

- packages/agents/planning/agent.py
- prompts/planning.md
- tests/unit/test_planning.py

What execute() must do:

1. Read research from input.context['research']
2. Query LightRAG for extra context using self.retrieve_from_memory()
3. Generate a structured outline with 3 to 7 sections
4. Return the outline as structured JSON

What to put in context:

```
context['outline'] = {
    'title': 'content title',
    'sections': [
        {'heading': 'Section 1', 'key_points': ['point 1']},
    ]
}
```

Your model:

```
nousresearch/hermes-3-405b-instruct:free
```

MEMBER 3 — WRITING AGENT

Your job:

Take the outline and write the full content. Clean prose only — no bullet points, no markdown headers.

Your files — you only touch these:

- packages/agents/writing/agent.py
- prompts/writing.md
- tests/unit/test_writing.py

What execute() must do:

1. Read outline from input.context['outline']
2. Query LightRAG using self.retrieve_from_memory()
3. Write full content following the outline exactly
4. Return clean prose between 500 and 2000 words

✗ **No markdown headers (no ##, no **)**

✗ **No bullet points in output**

✗ **Do not invent facts not in the outline**

✓ **Clean flowing paragraphs only**

What to put in context:

```
context['draft'] = {'content': 'full text here', 'word_count': 750}
```

Your model:

```
google/gemma-3-27b-it:free
```

MEMBER 4 — EDITING AGENT

Your job:

Review the draft, score its quality, and either approve it or send it back with specific improvement instructions.

Your files — you only touch these:

- packages/agents/editing/agent.py
- prompts/editing.md
- tests/unit/test_editing.py

What execute() must do:

1. Read draft from input.context['draft']
2. Score: coherence (0-10), relevance (0-10), completeness (0-10)
3. Calculate the average
4. If average < 7.0 — return status: retry with specific instructions
5. If average >= 7.0 — return status: success
6. After 3 retries — always pass (check input.context['edit']['retry_count'])

How to return a retry:

```
return AgentOutput(result='needs improvement', status='retry',
    metadata={'instructions': 'be more specific in section 2', ...})
```

What to put in context:

```
context['edit'] = {
    'scores': {'coherence': 8, 'relevance': 7, 'completeness': 6},
    'average': 7.0, 'passed': True, 'retry_count': 0
}
```

Your model:

```
google/gemma-3-27b-it:free
```

MEMBER 5 — OPTIMIZATION AGENT

Your job:

Polish the approved draft — improve tone, style, and flow without changing any facts.

Your files — you only touch these:

- packages/agents/optimization/agent.py
- prompts/optimization.md
- tests/unit/test_optimization.py

What execute() must do:

1. Read draft from input.context['draft']
2. Read edit scores from input.context['edit']
3. Refine tone, style, and length
4. Return final polished content — this is what the user sees

✗ **Do not change any facts**

✗ **Do not add new information**

✓ **Keep the same structure — just make it read better**

What to put in context:

```
context['final'] = {'content': 'polished text', 'word_count': 720, 'tone':  
'professional'}
```

Your model:

```
google/gemma-3-12b-it:free
```


MEMBER 6 — ORCHESTRATOR

Your job:

Build the entire system skeleton before anyone else starts. Then build the orchestrator that coordinates all agents.

You own all of these:

- packages/shared/ — schemas, model interface, base agent, settings
- packages/agents/orchestrator/ — orchestrator logic and pipeline
- packages/backend/ — FastAPI routes, services, database
- monitoring/ — Prometheus config, Grafana dashboards
- docker-compose.yml and docker-compose.dev.yml
- .github/workflows/ — CI/CD pipelines
- tests/integration/ — full pipeline tests

Phase 1 checklist — deliver this before anyone starts:

1. packages/shared/ complete: schemas.py, model.py, base_agent.py, settings.py
2. All 6 agent folders with dummy execute() returning hardcoded output
3. Orchestrator hub-spoke routing wired end to end
4. FastAPI running: /generate, /status, /result, /health, WebSocket
5. Redis job queue working
6. PostgreSQL + Alembic migrations for jobs and results
7. docker-compose.dev.yml running on Alibaba VPS
8. GitHub repo: all branches, protection rules, CI/CD configured
9. pre-commit hooks configured
10. README with exact setup steps per member
11. Each member's feature branch created and pushed

Acceptance criteria:

```
curl -X POST http://localhost:8000/generate
```

Must return a job that passes through all 6 dummy agents successfully.

Orchestrator rules:

- ✓ **Route in order: Research → Planning → Writing → Editing → Optimization**
- ✓ **On retry from Editing: create new task for Writing with improvement instructions**
- ✓ **On agent failure: log it, skip, continue pipeline**
- ✓ **Broadcast WebSocket update on every agent transition**
- ✗ **Never expose raw errors to the frontend — sanitize them**
- ✗ **Never let one failure stop the whole pipeline**

Questions?

Ask Member 6 first.

Try ADK Web debugging before asking.

The detailed architecture document has the full picture.